

Supermídia — Modelo-Alvo da Arquitetura

Documento de referência resultante da revisão de rumo de 2026-06-26. Conclusão: **a fundação atende ao propósito; não há reescrita, apenas extensões pontuais.**

Propósito do sistema

ERP de orçamentação para gráfica / comunicação visual, de **uso interno**. O orçamento é o **meio**: a partir dele compõe-se um **produto-template**, e um **pedido** é uma composição desses produtos. Os preços são **sugestivos** e podem ser ajustados no pedido.

As 5 camadas



1. Catálogo — pronto

Materia e Servico: nome, unidade, preco (= **custo unitário**). Sem mudança.

2. Cálculo — existe + 1 ajuste

Calculo modela a regra de consumo (área, perímetro, unidade). **Pendência: implementar UNIDADE_FIXA** no motor (hoje lança "não implementado") — cobre bastão, ponteira, grampo e o

"fixo".

3. Produto (template)

Composição de matérias/serviços (cada um amarrado a um `Calculo`). **Não carrega margem** — a margem é automática (ver camada 4).

Revisão 2026-06-29: a 1ª versão pôs markup MANUAL por produto (`markupAtacado / markupVarejo`); foi implementada e depois **descartada**. A empresa prefere com **margem automática** (fórmula do PHP, fruto de meses de teste). Esses campos serão **removidos** do produto, DTOs, mapper, motor e formulário.

4. Motor — margem automática por item

`ProdutoCalculoService` calcula o custo (separando `totalMateriais` e `totalServicos`) e aplica a margem automática **por item**:

- `margemAtacado = max(0,35 ; 1 - custoServiço/custoMaterial)` — material puxa margem, mão de obra não, piso 35%. **Sem material → 35%** (não divide).
- `preçoVarejo = preçoAtacado × 1,3846` (fator 1,8/1,3, 4 casas).
- Atacado vs varejo pela `Cliente.categoria` (REVENDA→atacado, FINAL→varejo).
- Preços em centavos. Constantes (0,35 e 1,3846) fixas por ora → futura **config global**.

Efêmero na fase de orçar (preview); congelado no `ItemVenda` ao criar a venda.

5. Venda (Orçamento / Ordem de Serviço) — NOVO

Uma única entidade `Venda` com `status` — não dois cadastros. O documento evolui de orçamento para OS por **mudança de status** (não há herança/subtipo). Na UI vira duas telas: "Orçamentos" e "Ordens de Serviço".

```

Venda
├ cliente          (→ define atacado/varejo via categoria)
├ status           (ORCAMENTO | ORDEM_SERVICO | CANCELADO; futuro: EM_PRODUCAO,
ENTREGUE)
├ dataCriacao      (base da validade de 15 dias do orçamento)
├ itens: [ItemVenda]
└ total

ItemVenda ("memória de cálculo" congelada / snapshot)
├ produtoId + produtoNome (qual template gerou)
├ altura, largura, quantidade
├ detalhamento[] (insumo, qtd calculada, custo unit. CONGELADO, subtotal)
├ custoTotal       (snapshot)
├ markupAplicado   (snapshot)
├ precoSugerido    (snapshot)
└ precoFinal       (editável – único override)

```

Imutabilidade: o produto-template aponta para o catálogo vivo; o `ItemVenda` guarda uma **fotografia** do cálculo. Mudança de preço no catálogo **não** altera venda já feita. A memória de cálculo é auditável e reproduzível.

Por que estado e não herança (padrão Pessoa): Pessoa→Cliente/Fornecedor/etc. são papéis que *coexistem* num mesmo indivíduo; orçamento e OS são *fases sequenciais* do mesmo documento. Estado (campo `status`) preserva identidade e histórico na conversão; subtipo exigiria recriar a linha (perdendo ambos).

Validade do orçamento: 15 dias, calculada como estado *derivado* (`dataCriacao + 15d < hoje`), sem rotina/agendador. Ao abrir um orçamento vencido: avisar e oferecer

"**Recalcular/Renovar**" (reprocessa com preços atuais, regrava snapshot, reinicia os 15 dias) ou "**Cancelar**". Futuramente o "15" migra para uma configuração global do sistema.

Cabeçalho adiado: forma de pagamento, forma de entrega, prazo de entrega e desconto no pedido serão modelados depois.

Decisões fixadas (2026-06-26)

Tema	Decisão
Margem	Automática por item (revisão 2026-06-29): $\max(0, 35; 1 - \text{custoServiço}/\text{custoMaterial})$, sem material → 35%; varejo = atacado × 1,3846. Constantes → futura config global. (Markup manual por produto foi descartado.)
Atacado × Varejo	Definido pela categoria do cliente (REVENDA→atacado, FINAL→varejo)
Override na venda	Apenas o precoFinal do item. Quantidade de insumo é sempre auto-calculada
"Fixo" (R\$5)	Por item (não por venda)
Entidade da venda	Nome interno Venda (UI: "Orçamentos"/"Ordens de Serviço"); um registro + status , sem herança
Validade do orçamento	15 dias (derivado); vencido → "Recalcular/Renovar" ou "Cancelar"
Bastão	Tratado por unidade (não por comprimento/tabela)
Regras "espertas" (#3)	Fora de escopo por ora: condicional por dimensão, seleção por tabela, sobretaxa de peça pequena
permiteOverrideParametro	Flag fica dormente nesta fase (mantida, não usada)

Origem da lógica de preço

Decodificada dos scripts PHP legados em `calculadoras/` (`smlona`, `smbanner`, `smimpressaocomrecorte`, `smimpressaosemrecorte + precos.php`). Cada script é um produto-template hardcoded; o sistema novo os torna configuráveis como dado. A margem da empresa segue a **fórmula automática do PHP** ($\max(0, 35; 1 - \text{custoServiço}/\text{custoMaterial})$) e varejo = atacado × 1,3846) — validada por meses de uso. A tentativa de markup manual por produto foi descartada (ver revisão 2026-06-29 acima).

Ordem de implementação

1. **UNIDADE_FIXA** no motor (+ testes) — desbloqueia os produtos legados; pequeno e isolado.
2. **Markup no Produto** + seleção por `Cliente.categoria` no motor — fecha a precificação de orçamento.
3. **Validar** reproduzindo `smlona / smbanner` como produtos-template (prova real da arquitetura).
4. **Camada Venda** (`Venda` + `ItemVenda` com snapshot; status ORCAMENTO/ORDEN_SERVICO/CANCELADO).